

Συστήματα Ανάκτησης Πληροφοριών (INF248)

Αναφορά Προγραμματιστικής Εργασίας – Φάση 1: Baseline

Ρήγας Σάσσαλος

Αρ. Μητρώου: 3220178

Στην πρώτη φάση της εργασίας, υλοποίησα ένα βασικό σύστημα ανάκτησης πληροφοριών (baseline) χρησιμοποιώντας την μηχανή αναζήτησης *ElasticSearch*. Στόχος είναι η εφαρμογή κλασικών μοντέλων ανάκτησης στην συλλογή κειμένων IR2025, η οποία αποτελείται από 18.316 κείμενα. Η αξιολόγηση του συστήματος πραγματοποιήθηκε με το εργαλείο *trec_eval*.

Για την υλοποίηση χρησιμοποίησα την γλώσσα προγραμματισμού *Python* και την βιβλιοθήκη *elasticsearch*. Αρχικά, διάβασα το αρχείο *documents.csv* το οποίο περιέχει τα έγγραφα της συλλογής. Δημιούργησα ένα ευρετήριο με την ονομασία *ir2025_documents*.

```
# Create the index
client.indices.create(index=index_name, body=settings)
print(f"Index '{index_name}' created.")
```

✓ 0.9s Python

Index 'ir2025_documents' created.

Χρησιμοποίησα τον *English Analyzer* για την ανάλυση του κειμένου, ο οποίος περιλαμβάνει διαδικασίες όπως το *tokenization* και το *stemming* που αφαιρεί καταλήξεις. Ως συνάρτηση ομοιότητας επιλέχθηκε η *BM25Similarity*. Η εισαγωγή των δεδομένων έγινε μαζικά χρησιμοποιώντας την κλάση *helpers.bulk*.

```
# Define settings: English Analyzer and BM25 Similarity
settings = {
    "settings": {
        "number_of_shards": 1,
        "analysis": {
            "analyzer": {
                "default": {
                    "type": "english" # Handles stemming, stopwords automatically
                }
            },
            "similarity": {
                "default": {
                    "type": "BM25" # Standard probabilistic model
                }
            }
        },
    },
    "mappings": {
        "properties": {
            "text": { "type": "text" }, # The content of the document
            "doc_id": { "type": "keyword" } # The document ID
        }
    }
}
```

✓ 0.0s Python

Αφού ολοκληρώθηκε η δημιουργία του ευρετηρίου, επεξεργάστηκα το αρχείο *queries.csv*. Για κάθε ερώτημα, εκτέλεσα μια αναζήτηση τύπου *match* στο πεδίο του ευρετηρίου. Σύμφωνα με τις απαιτήσεις της άσκησης, η διαδικασία αναζήτησης παραμετροποιήθηκε ώστε να εξάγει τα *k* πρώτα ανακτηθέντα κείμενα για τις τιμές *k=20,30* και *50*.

Για την παραγωγή του τελικού αρχείου αποτελεσμάτων χρησιμοποίησα την τιμή $k=50$ για κάθε ερώτημα. Η επιλογή αυτή έγινε καθώς το σύνολο των 50 κορυφαίων αποτελεσμάτων περιλαμβάνει και τις λίστες για $k=20$ και $k=30$, επιτρέποντας στο εργαλείο `trec_eval` να υπολογίσει με ακρίβεια τις μετρικές σε όλα τα ζητούμενα επίπεδα. Τα αποτελέσματα αποθηκεύτηκαν σε ένα αρχείο κειμένου (*results.txt*) στην μορφή που απαιτεί το `trec_eval` (*query_id Q0 doc_id rank score run_name*)

```
rigas — docker run -p 9200:9200 -e discovery.type=single-node -e xpa...
Last login: Thu Jan  8 18:53:28 on ttys007
rigas@RigasMacbookAir ~$ docker run -p 9200:9200 -e "discovery.type=single-node" ]
-e "xpack.security.enabled=false" docker.elastic.co/elasticsearch/elasticsearch:
8.11.1
Jan 08, 2026 5:29:53 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
WARNING: COMPAT locale provider will be removed in a future release
{"@timestamp":"2026-01-08T17:29:54.062Z", "log.level": "INFO", "message":"Java v
ector incubator API enabled; uses preferredBitSize=128", "ecs.version": "1.2.0",
"service.name":"ES_ECS", "event.dataset":"elasticsearch.server", "process.thread.n
ame":"main", "log.logger":"org.apache.lucene.internal.vectorization.PanamaVectori
zationProvider", "elasticsearch.node.name":"577cd98f1e9e", "elasticsearch.cluster.
name":"docker-cluster"}
{"@timestamp":"2026-01-08T17:29:54.279Z", "log.level": "INFO", "message":"versio
n[8.11.1], pid[67], build[docker/6f9ff581fbcde658e6f69d6ce03050f060d1fd0c/2023-1
1-11T10:05:59.421038163Z], OS[Linux/6.12.54-linuxkit/aarch64], JVM[Oracle Corpora
tion/OpenJDK 64-Bit Server VM/21.0.1/21.0.1+12-29]", "ecs.version": "1.2.0", "se
rvice.name":"ES_ECS", "event.dataset":"elasticsearch.server", "process.thread.name
":"main", "log.logger":"org.elasticsearch.node.Node", "elasticsearch.node.name":"5
77cd98f1e9e", "elasticsearch.cluster.name":"docker-cluster"}
{"@timestamp":"2026-01-08T17:29:54.279Z", "log.level": "INFO", "message":"JVM ho
me [/usr/share/elasticsearch/jdk], using bundled JDK [true]", "ecs.version": "1.
2.0", "service.name":"ES_ECS", "event.dataset":"elasticsearch.server", "process.thr
ead.name":"main", "log.logger":"org.elasticsearch.node.Node", "elasticsearch.node.
name":"577cd98f1e9e", "elasticsearch.cluster.name":"docker-cluster"}
```

Η αξιολόγηση έγινε συγκρίνοντας τις απαντήσεις του συστήματος μας με τις σωστές απαντήσεις από το αρχείο *qrels.txt*. Χρησιμοποιήσαμε το εργαλείο `trec_eval` για τον υπολογισμό των εξής μετρικών:

- MAP (Mean Average Precision)
- Precision@k για $k = 5, 10, 15, 20$

```
rigas@RigasMacbookAir IR_Phase1$ ./trec_eval-9.0.7/trec_eval -m map -m P.5,10,15,]
20 data/qrels.txt results/phase1_results.txt
map                all      0.7446
P_5                all      0.8600
P_10               all      0.7300
P_15               all      0.6867
P_20               all      0.6200
```

MAP	0.7446
P@5	0.8600
P@10	0.7300
P@15	0.6867
P@20	0.6200

Παρατηρώντας τα αποτελέσματα, βλέπουμε ότι το μοντέλο BM25 σε συνδυασμό με τον English Analyzer παρέχει ένα αξιόλογο σημείο αναφοράς.

- **Precision:** Παρατηρούμε ότι η ακρίβεια είναι υψηλότερη στα πρώτα αποτελέσματα ($k = 5$) και τείνει να μειώνεται ελαφρώς όσο αυξάνεται το k . Αυτό δείχνει ότι η συνάρτηση κατάταξης τοποθετεί τα σχετικά έγγραφα ψηλά στην λίστα.

- **Επιρροή Analyzer:** Η χρήση του English Analyzer βοήθησε σημαντικά γιατί επέτρεψε την ταύτιση όρων παρά τις διαφορές τους βελτιώνοντας το recall.
- **Περιορισμοί:** Το σύστημα βασίζεται αποκλειστικά σε λεξική σύγκριση. Αυτό σημαίνει ότι αν ένα έγγραφο χρησιμοποιεί συνώνυμα δεν θα ανακτηθεί.